

7.1 Introduction – Easy Explanation

Why do we need random numbers in simulation?

- When we simulate real-life systems (like queues in banks or inventory in stores), some things are **random**:
 - Customers arrive at random times
 - Service times vary
 - Demand sizes differ
- To model this, we need numbers that act **randomly**. Example: If the time between customers is random, we “draw” it from a probability distribution (like exponential or Erlang).

Random variables vs random variates

- In math, a **random variable** is a special function with strict rules.
- In simulation, we usually just **generate random numbers** to represent random events.
- So we use the term **random variates** instead of “random variables” – it’s more correct.

Uniform distribution $U(0,1)$

- The simplest random numbers are between **0 and 1** (called **uniform distribution**, written $U(0,1)$).
- Why important? Because: We can transform these numbers into any other distribution (normal, binomial, gamma, etc.). We can simulate complex processes (like Poisson arrivals) using these simple numbers.

History of generating random numbers

- Long ago, people used **physical random methods**:
 - Casting lots, dice, cards, or drawing balls from urns.
 - Lotteries still use similar methods.
- Early 1900s: **mechanical machines** made random numbers faster.
- Late 1930s: Spinning disks generated **tables of 100,000 digits**.
- 1940s–50s: Electronic machines (like **ERNIE**) generated numbers for lotteries.
- Other creative ways: digits of π , gamma rays, phone books, census data.

Computers and random numbers

- Directly connecting a random machine to a computer has problems:
 - Can't reproduce the same numbers later (needed for debugging).
 - Reading big tables of numbers is **slow and memory-heavy**.
- Solution: **arithmetic or formula-based generators** (pseudo-random):
 - Each new number depends on previous numbers via a formula.
 - First famous method: **midsquare method** by von Neumann.

Midsquare method – Simple example

1. Start with a 4-digit number (Z_0) (like 7182).
2. Square it → get 8-digit number.
3. Take **middle 4 digits** → next number (Z_1).
4. Add decimal in front → get $U(0,1)$ random number.
5. Repeat → get more random numbers.

Problem:

- It often **degenerates to zero** quickly.
- Not truly random because formula **predicts next number**.
- So it's more like a **recipe** than real randomness.

Pseudorandom numbers

- Arithmetic generators are **not truly random**, but can behave like random numbers if:
 - They pass statistical tests for uniformity and independence.
- A “good” generator should:
 1. **Be uniform and independent** – numbers shouldn't repeat patterns.
 2. **Be fast and use little memory**.
 3. **Allow reproducibility** – can generate same sequence for debugging or comparison.
 4. **Support multiple streams** – e.g., separate streams for arrivals vs service times in a queue.
 5. **Be portable** – same results on different computers.

Testing random-number generators

- Modern generators meet points 2–5 easily.
- But **some fail uniformity or independence** → bad for simulations.
- We must test them using statistical methods (see Sec. 7.4).

Summary

- Random-number generation is **complex**:
 - Mixes **math** (algebra, number theory) with **computers** (programming, hardware).
- Historical methods → physical machines → tables → formulas (arithmetic).
- For simulation, **pseudo-random numbers** from uniform $[0,1]$ are **enough if tested well**.
- They can be converted to any needed distribution for real simulations.

7.2 Linear Congruential Generators (LCGs)

What is an LCG?

- LCG = a **formula-based way** to generate pseudo-random numbers on a computer. Introduced by **Lehmer (1951)**.

Important points about LCGs

- The sequence of (Z_i) is **completely determined** by the four parameters (m, a, c, Z_0) .
 - So, not truly random – it's **pseudo-random**.
 - But if we choose the parameters carefully, (U_i) will behave like **independent random numbers**.
- LCG numbers can only take **specific values**: $(0/m, 1/m, 2/m, \dots, (m-1)/m)$.
 - If (m) is large (like (10^9)), there are billions of possible values → practically looks random. Example of a small LCG
- Let: $(m=16), (a=5), (c=3), (Z_0=7)$ Generate a few numbers:

i	Z_i	U_i
0	7	—
1	6	0.375
2	1	0.063
3	8	0.500
4	11	0.688
...	...	...

- Observation: **After 16 numbers, the sequence repeats** → this is called **looping or cycling**.

Period of an LCG

- **Period** = length before the sequence repeats.
- Maximum period = (m) .
 - If period = $(m) \rightarrow$ **full period**.
 - Full-period generators ensure all numbers 0 to $(m-1)$ appear **once per cycle**.
- Long periods are needed because simulations may use **millions of random numbers**.

How to get a full-period LCG

According to **Hull and Dobell (1962)**, full period occurs if:

1. (c) and (m) are **relatively prime** (no common divisors except 1).
 2. If a prime number (q) divides (m) , it must also divide $(a-1)$.
 3. If 4 divides (m) , 4 must divide $(a-1)$.
- Choosing good (a, c, m) = essential for a **good generator**.

Streams of random numbers

- We often need **multiple independent sequences** (streams).
- Set each stream with a **different starting seed**.
- Example:
 - Stream 1: $(Z_0 = 7) \rightarrow$ first 1,000,000 numbers
 - Stream 2: $(Z_{\{1,000,001\}}) \rightarrow$ next 1,000,000 numbers
- Streams prevent **correlation** between random numbers used for different parts of simulation.

7.3 Other Kinds of Generators – Easy Explanation

Why look for alternatives to LCGs?

- **LCGs** are the most common and well-understood generators.
- But they sometimes have **shorter periods** or **weaker statistical properties**.
- So researchers made **other generators** to: Increase period length, Improve randomness quality. Example of a bad old alternative: **midsquare method** (see Sec. 7.1) $\rightarrow$ not recommended.

1. Generalized congruential generators
2. Quadratic congruential generators
3. Multiple recursive generators (MRGs)
4. Fibonacci generator

Theorem 7.1 – Full Period LCG

A linear congruential generator (LCG)

$$Z_i = (a Z_{i-1} + c) \bmod m$$

will have **full period** (i.e., the sequence goes through all integers from 0 to (m-1) before repeating) **if and only if** the following three conditions are satisfied:

1. (c) and (m) are **relatively prime** (no common factors other than 1).
2. If a prime number (q) divides (m), then (q) also divides (a - 1).
3. If 4 divides (m), then 4 also divides (a - 1).

Meaning: If these conditions are met, the LCG will generate the **longest possible cycle** of numbers without repetition.

Example 7.1 – Midsquare Method

Goal: Generate random numbers (U_i) in [0,1] using the midsquare method. Steps:

1. Start with a 4-digit number (Z_0) (seed).
2. Square it → get an 8-digit number (add leading zeros if needed).
3. Take the **middle 4 digits** → this is (Z_1).
4. Put a decimal point in front → ($U_1 = Z_1/10000$).
5. Repeat to get more numbers ($Z_2, Z_3, \dots$)

Example: Start with ($Z_0 = 7182$)

i	Z_i	U_i
0	7182	—
1	5811	0.5811
2	7677	0.7677
3	9363	0.9363
4	6657	0.6657
5	3156	0.3156

Note: Seems random, but eventually degenerates → not reliable for serious simulations.

Example 7.2 – Linear Congruential Generator (LCG)

Given: LCG with

$m = 16, a = 5, c = 3, Z_0 = 7$

Formula: $(Z_i = (5 Z_{i-1} + 3) \bmod 16)$

Generate Z_i and $U_i = Z_i/16$:

i	Z_i	U_i
0	7	—
1	6	0.375
2	1	0.063
3	8	0.500
4	11	0.688
5	10	0.625
6	5	0.313
7	12	0.750
8	15	0.938
9	14	0.875
10	9	0.563
11	0	0.000
12	3	0.188
13	2	0.125
14	13	0.813
15	4	0.250
16	7	0.438
17	6	0.375
18	1	0.063
19	8	0.500

Observation:

- Sequence **repeats after 16 numbers** → period = 16 (full period).
- Shows how LCG generates predictable but pseudo-random numbers.